CYKL - PLANO TÉCNICO, DESENVOLVIMENTO E VALIDAÇÃO

Projeto: CYKL
Documento: Plano Técnico, Desenvolvimento e Validação
Versão: 3.0
Disciplina: Projeto Integração Social IV
Professor(a): Pricylla Cavalcante
Equipe:

1. Camila Rebeca
2. Enrico Reno
3. Gabriel Lucas
4. Guilherme Enrique
5. José Miguel
6. Leonardo Rafael
7. Samuel Victor

Recife, 2026

1. Objetivo

Este documento define as diretrizes técnicas e operacionais para o desenvolvimento do MVP do CYKL, incluindo arquitetura, tecnologias, estrutura de dados, fluxos, divisão da equipe, cronograma, riscos e testes.

A prioridade será entregar uma aplicação funcional, estável e demonstrável dentro do prazo disponível.

2. Diretrizes de Desenvolvimento

- Priorizar o fluxo completo de doação.
- Utilizar tecnologias conhecidas pela equipe.
- Evitar integrações e serviços desnecessários.
- Manter o código organizado e compreensível.
- Desenvolver e integrar funcionalidades gradualmente.
- Validar as operações de dados e permissões.
- Não desenvolver funcionalidades fora do escopo do MVP.
- Preservar uma versão executável durante todo o desenvolvimento.

Diretriz principal: priorizar poucas funcionalidades funcionando corretamente, em vez de muitas funcionalidades incompletas.

3. Arquitetura Proposta

O CYKL será desenvolvido como uma aplicação web, composta por uma interface de usuário e serviços responsáveis pela autenticação e persistência dos dados.

3.1. Tecnologias recomendadas

Camada	Tecnologia	Finalidade
Front-end	React.js + Vite	Interface da aplicação
Linguagem	TypeScript	Tipagem e organização
Estilização	Tailwind CSS	Construção visual
Ícones	Lucide React	Elementos visuais
Banco de dados	PostgreSQL	Persistência
Backend gerenciado	Supabase	Autenticação e acesso ao banco
Versionamento	Git e GitHub	Controle de código
Design	Figma	Protótipos

Hospedagem	Vercel ou equivalente	Publicação

Para o prazo de sete dias, será utilizado um serviço gerenciado, como Supabase, evitando desenvolver uma API e uma infraestrutura completas do zero.

3.2. Organização do projeto

```
cykl/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── ui/
│   │   ├── layout/
│   │   └── items/
│   ├── pages/
│   ├── routes/
│   ├── services/
│   ├── hooks/
│   ├── contexts/
│   ├── types/
│   ├── utils/
│   ├── styles/
│   ├── lib/
│   ├── App.tsx
│   └── main.tsx
├── .env.example
├── package.json
└── README.md
```

A estrutura poderá ser adaptada conforme a implementação. Não é necessário criar arquivos ou pastas sem utilização real.

4. Modelagem Inicial do Banco de Dados

O modelo deverá permitir cadastrar diferentes categorias de itens sem criar uma tabela específica para cada tipo de produto.

4.1. Tabela: users

Armazena os dados básicos dos usuários.

Campo	Tipo sugerido	Descrição
id	UUID	Identificador
name	VARCHAR	Nome

email	VARCHAR	E-mail
phone	VARCHAR	Telefone
city	VARCHAR	Cidade
neighborhood	VARCHAR	Bairro
created_at	TIMESTAMP	Data de cadastro

A autenticação poderá ser gerenciada pelo Supabase Auth, utilizando o identificador do usuário autenticado.

4.2. Tabela: categories

Armazena as categorias de itens disponíveis.

Campo	Tipo sugerido	Descrição
id	UUID	Identificador
name	VARCHAR	Nome da categoria
description	TEXT	Descrição

Categorias iniciais:

- Materiais de construção.
- Móveis.
- Eletrodomésticos.
- Roupas e calçados.
- Cobertores e itens de cama, mesa e banho.
- Outros.

4.3. Tabela: items

Armazena os itens disponibilizados para doação.

Campo	Tipo sugerido	Descrição
id	UUID	Identificador
owner_id	UUID	Usuário responsável
category_id	UUID	Categoria
name	VARCHAR	Nome do item

description	TEXT	Descrição
quantity	DECIMAL	Quantidade
unit	VARCHAR	Unidade, quando aplicável
condition	VARCHAR	Estado de conservação
city	VARCHAR	Cidade
neighborhood	VARCHAR	Bairro
image_url	TEXT	Imagem, se implementada
status	VARCHAR	Estado do anúncio
created_at	TIMESTAMP	Data de cadastro
updated_at	TIMESTAMP	Data de atualização

4.4. Tabela: requests

Armazena as solicitações realizadas.

Campo	Tipo sugerido	Descrição
id	UUID	Identificador
item_id	UUID	Item solicitado
requester_id	UUID	Usuário solicitante
message	TEXT	Mensagem opcional
status	VARCHAR	Estado da solicitação
created_at	TIMESTAMP	Data da solicitação
updated_at	TIMESTAMP	Data de atualização

A quantidade não será armazenada na tabela de solicitações, pois, no MVP, cada solicitação deverá corresponder ao lote completo anunciado. A quantidade ficará registrada exclusivamente no anúncio.

4.5. Relacionamentos

- Um usuário poderá cadastrar vários itens.
- Uma categoria poderá estar associada a vários itens.
- Um item pertencerá a um único usuário responsável.

- Um item poderá receber várias solicitações pendentes ou recusadas ao longo do tempo.
- Um item poderá possuir somente uma solicitação aprovada por vez.
- Uma solicitação pertencerá a um item e a um usuário solicitante.
- Um usuário poderá cadastrar itens e também realizar solicitações.

4.6. Flexibilidade das categorias

O cadastro será genérico, permitindo que diferentes tipos de itens compartilhem os mesmos campos principais.

Exemplos:

- Um material de construção poderá utilizar quantidade, unidade e conservação.
- Um móvel poderá utilizar quantidade, descrição e conservação.
- Uma roupa poderá utilizar quantidade, tamanho informado na descrição e conservação.
- Um eletrodoméstico poderá ter marca e informações de funcionamento descritas no campo de descrição.

No MVP, não será necessário criar atributos específicos, filtros avançados ou formulários diferentes para cada categoria.

Caso novas categorias sejam incorporadas futuramente, poderão ser adicionados campos específicos conforme a necessidade.

4.7. Controle de disponibilidade

Para simplificar o desenvolvimento, cada anúncio será tratado como um lote único.

Ao aprovar uma solicitação:

1. O anúncio passará para o estado reservado.
2. Nenhuma outra solicitação poderá ser aprovada para o mesmo anúncio.
3. A quantidade do lote não será dividida entre diferentes solicitantes.
4. A solicitação aprovada será vinculada ao anúncio.
5. Após a confirmação do recebimento, o anúncio será marcado como concluído.

A solicitação deverá corresponder à totalidade da quantidade disponibilizada no anúncio.

A aprovação e a reserva deverão ser realizadas de forma consistente no banco de dados, evitando que duas solicitações sejam aprovadas simultaneamente.

5. Fluxos Operacionais

5.1. Cadastro e acesso

1. O visitante acessa a plataforma.

2. Seleciona a opção de cadastro.
3. Informa os dados necessários.
4. O sistema valida as informações.
5. A conta é criada.
6. O usuário realiza o login.

5.2. Disponibilização de item

1. O usuário acessa sua conta.
2. Seleciona a opção de cadastrar item.
3. Informa as características do item.
4. Seleciona a categoria.
5. O sistema valida os dados.
6. O item é armazenado.
7. O anúncio aparece no catálogo como disponível.

5.3. Consulta e solicitação

1. O usuário acessa o catálogo.
2. Pesquisa ou filtra os itens.
3. Seleciona um anúncio.
4. Consulta suas informações.
5. Solicita o item.
6. O sistema registra a solicitação como pendente.
7. O responsável pelo anúncio visualiza a solicitação.

5.4. Aprovação e conclusão

1. O responsável consulta as solicitações recebidas.
2. Aprova ou recusa uma solicitação.
3. Se aprovada, o anúncio passa para reservado.
4. As informações necessárias para combinar a retirada são disponibilizadas aos envolvidos.
5. As partes combinam a retirada.
6. O solicitante confirma o recebimento.
7. A solicitação passa para concluída.
8. O anúncio deixa de aceitar novas solicitações.

6. Telas Previstas

1. **Página inicial:** apresentação do CYKL e acesso às funcionalidades.
2. **Cadastro:** criação de conta.
3. **Login:** autenticação.
4. **Catálogo:** listagem e pesquisa de itens.
5. **Detalhes do item:** características e opção de solicitar.
6. **Cadastro de item:** formulário de disponibilização.
7. **Meus anúncios:** gerenciamento dos itens cadastrados.

8. **Minhas solicitações:** acompanhamento das solicitações realizadas.
9. **Solicitações recebidas:** gerenciamento das solicitações dos anúncios do usuário.
10. **Perfil:** visualização e edição dos dados básicos.

As telas deverão reutilizar componentes sempre que possível, reduzindo o esforço de desenvolvimento.

7. Segurança e Privacidade

A implementação deverá observar os seguintes cuidados:

- Utilizar autenticação segura.
- Não armazenar senhas em texto puro.
- Restringir a edição e exclusão de anúncios aos seus responsáveis.
- Restringir a aprovação e recusa aos responsáveis pelos anúncios.
- Impedir alterações indevidas em solicitações de terceiros.
- Validar operações também no backend ou nas políticas do banco.
- Não expor publicamente informações pessoais desnecessárias.
- Utilizar variáveis de ambiente para configurações e chaves privadas.
- Configurar políticas de segurança do banco, como Row Level Security (RLS), caso seja utilizado Supabase.
- Disponibilizar informações de contato somente quando necessárias para a realização da doação.
- Coletar apenas os dados pessoais necessários para o funcionamento da plataforma.

8. Cronograma de Desenvolvimento - 7 Dias

Dia 1 - Preparação

- Validar o escopo.
- Criar o repositório.
- Configurar o ambiente.
- Criar o projeto.
- Configurar banco e autenticação.
- Definir contratos de dados.
- Dividir as tarefas.

Entrega: estrutura inicial pronta e ambiente funcionando.

Dia 2 - Usuários e autenticação

- Implementar cadastro.
- Implementar login e logout.
- Criar perfil básico.
- Proteger rotas.
- Testar autenticação.

Entrega: usuário consegue criar conta e acessar a plataforma.

Dia 3 - Cadastro de itens

- Criar formulário.
- Implementar persistência.
- Implementar edição.
- Criar listagem dos próprios anúncios.
- Implementar exclusão com validação.

Entrega: usuário consegue disponibilizar e gerenciar itens.

Dia 4 - Catálogo

- Implementar listagem.
- Criar detalhes dos itens.
- Implementar pesquisa.
- Implementar filtro por categoria.
- Exibir disponibilidade.

Entrega: usuários conseguem encontrar itens disponíveis.

Dia 5 - Solicitações

- Implementar solicitação.
- Criar listagem de solicitações recebidas.
- Implementar aprovação e recusa.
- Implementar reserva.
- Criar visualização das solicitações realizadas.

Entrega: fluxo de solicitação e decisão funcionando.

Dia 6 - Integração e testes

- Integrar funcionalidades.
- Implementar confirmação de recebimento.
- Corrigir erros de persistência.
- Testar permissões.
- Validar regras de negócio.
- Ajustar responsividade.

Entrega: fluxo completo de doação funcionando.

Dia 7 - Estabilização e entrega

- Executar testes finais.
- Corrigir erros críticos.
- Verificar segurança e permissões.

- Publicar a aplicação.
- Preparar demonstração.
- Atualizar README.

Entrega: MVP funcional, publicado e pronto para apresentação.

Regra de contingência: caso ocorram atrasos, retirar funcionalidades secundárias e preservar o fluxo principal. O último dia deverá ser reservado à estabilização, e não ao início de novos módulos.

9. Plano de Testes

9.1. Testes funcionais

- Criar conta com dados válidos.
- Tentar cadastrar e-mail já utilizado.
- Realizar login com credenciais corretas e incorretas.
- Cadastrar item.
- Tentar cadastrar item sem campos obrigatórios.
- Pesquisar item por nome.
- Filtrar por categoria.
- Solicitar item disponível.
- Tentar solicitar item próprio.
- Tentar solicitar item reservado.
- Aprovar solicitação.
- Recusar solicitação.
- Cancelar solicitação pendente.
- Confirmar recebimento.
- Consultar histórico.

9.2. Testes de autorização

- Usuário não poderá editar anúncios de terceiros.
- Usuário não poderá excluir anúncios de terceiros.
- Usuário não poderá aprovar solicitações de anúncios de terceiros.
- Usuário não poderá visualizar dados privados desnecessários.
- Usuário não poderá modificar estados de solicitações sem autorização.

9.3. Testes de interface

- Verificar navegação entre as telas.
- Testar em computador e celular.
- Verificar formulários e mensagens de erro.
- Verificar catálogo vazio.
- Verificar ausência de solicitações.
- Verificar apresentação dos anúncios e estados.

10. Gestão de Riscos

Risco	Impacto	Mitigação
Atraso no desenvolvimento	Alto	Priorizar o fluxo principal
Dificuldade técnica	Alto	Utilizar ferramentas conhecidas
Problemas de integração	Alto	Definir contratos de dados no primeiro dia
Erros de persistência	Alto	Testar operações e manter backups
Escopo excessivo	Alto	Respeitar as funcionalidades do MVP
Falta de comunicação	Médio	Realizar alinhamentos diários
Falhas de autenticação	Alto	Utilizar serviço gerenciado
Cadastro de itens inadequados	Médio	Definir categorias permitidas e regras claras
Informações incorretas nos anúncios	Médio	Responsabilizar o usuário pelas informações
Problemas de retirada	Médio	Informar que a logística será combinada entre os usuários
Exposição de dados pessoais	Alto	Aplicar controle de acesso e minimização de dados
Aprovação simultânea de solicitações	Alto	Implementar validação transacional no banco

12. Organização e Versionamento

O desenvolvimento do CYKL utilizará o Git para controle de versão e o GitHub para hospedagem do código-fonte e colaboração entre os integrantes da equipe.

Repositório oficial: CYKL-Frontend

12.1. Estratégia de Branches — Git Flow

O projeto seguirá o Git Flow como estratégia de ramificação, organizando o desenvolvimento em três níveis principais:

main → Produção: código estável, validado e pronto para deploy.
dev → Homologação e testes: integração das funcionalidades.
feature/* → Desenvolvimento individual de novas funcionalidades.

As branches `feature/*` deverão ser criadas a partir da `dev` e utilizadas para implementar funcionalidades ou realizar alterações específicas. Após a conclusão, as alterações deverão ser enviadas para a `dev` por meio de Pull Requests.

12.2. Fluxo de Trabalho da Equipe

O fluxo de integração seguirá as seguintes etapas:

feature/* (desenvolvimento individual)
↓
Pull Request
↓
dev (revisão, integração e testes)
↓
Validação final
↓
Pull Request
↓
main (produção)

A branch `dev` será utilizada para reunir e validar as funcionalidades desenvolvidas pela equipe. Após a validação, as alterações poderão ser integradas à `main`, que deverá permanecer estável e conter apenas versões aprovadas do sistema.

12.3. Permissões e Regras de Integração

Ação	Permissão
Push direto na `main`	Não permitido
Push direto na `dev`	Não permitido
Criar branches `feature/*`	Todos os contribuidores
Abrir Pull Requests	Todos os contribuidores
Aprovar Pull Requests	Tech Leader e revisores designados
Integrar alterações na `main`	Mediante aprovação e validação final

12.4. Convenções de Desenvolvimento

Para manter a organização e a qualidade do código, deverão ser adotadas as seguintes práticas:

- Utilizar nomes descritivos para branches, seguindo o padrão `feature/nome-da-funcionalidade` ou `fix/descricao-do-problema`.
- Criar commits objetivos, descrevendo claramente as alterações realizadas.
- Manter as branches de desenvolvimento atualizadas em relação à `dev`.
- Utilizar Pull Requests para revisar e integrar todas as alterações nas branches `dev` e `main`.
- Verificar o funcionamento da aplicação antes de solicitar a integração.
- Evitar alterações simultâneas nos mesmos arquivos sempre que possível.
- Resolver conflitos antes da conclusão dos Pull Requests.
- Manter a branch `main` estável e a aplicação executável durante todo o desenvolvimento.

12. Critérios de Entrega

A entrega do MVP deverá contemplar:

- Aplicação web funcional.
- Cadastro e autenticação.
- Perfil básico.
- Cadastro e gerenciamento de itens.
- Catálogo e detalhes.
- Pesquisa e filtro por categoria.
- Solicitação de itens.
- Aprovação e recusa.
- Controle de disponibilidade.
- Histórico básico.
- Confirmação de recebimento.
- Interface responsiva.
- Persistência dos dados.
- Controle de acesso.
- Repositório organizado.
- README com instruções básicas.

13. Evolução Futura

Após a validação do MVP, poderão ser consideradas:

- Vendas a preços acessíveis.
- Pagamentos e comissões.
- Publicidade e anúncios patrocinados.
- Planos empresariais.
- Gestão de múltiplos usuários por organização.
- Upload e gerenciamento avançado de imagens.
- Filtros específicos por categoria.
- Chat e notificações.
- Geolocalização e busca por proximidade.

- Avaliações e reputação.
- Painel administrativo e moderação.
- Indicadores de impacto social e ambiental.
- Parcerias com empresas, instituições e poder público.
- Gestão logística e integração com transportadoras.
- Distribuição de lotes entre múltiplos solicitantes.

A evolução deverá ocorrer conforme os resultados de utilização e validação do MVP.